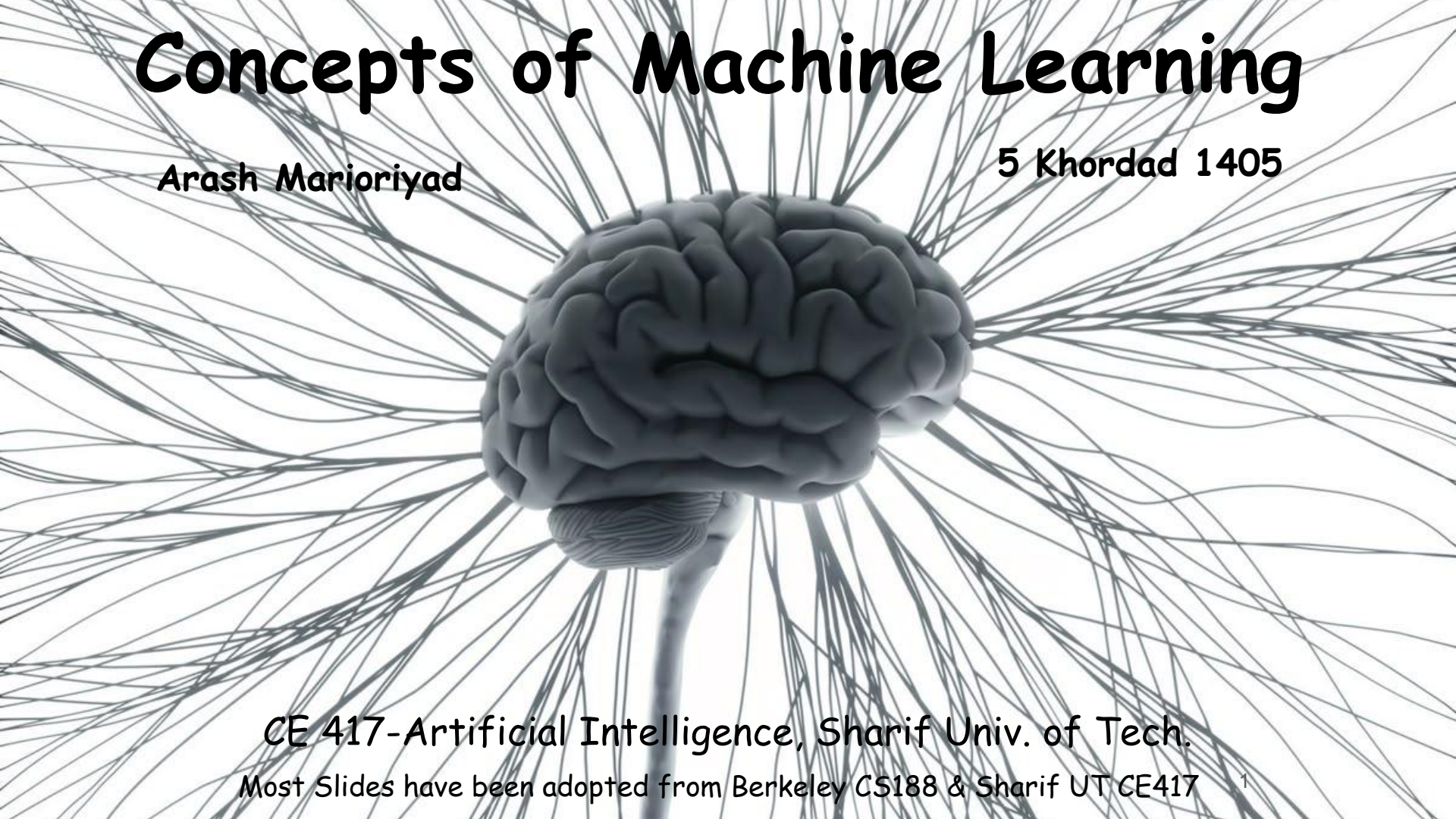
Concepts of Machine Learning

Arash Marioriyad

5 Khordad 1405



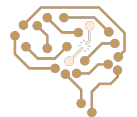
CE 417-Artificial Intelligence, Sharif Univ. of Tech.

Most Slides have been adopted from Berkeley CS188 & Sharif UT CE417

Up To Now

- Learning Parameters From Data
- Naive Bayes
  - Parameters: Conditional Probability Tables (CPTs)
- Decision Tree
  - Parameters: Nodes of the decision tree (features)

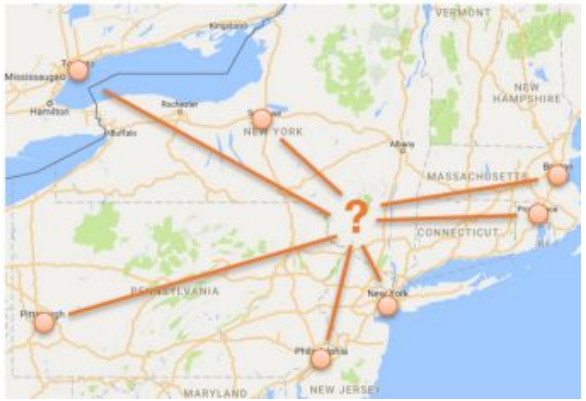
Continuous Search (Optimization)



Example: Weber Point

- Given a collection of cities (assume on 1D line or 2D plane) how can we find the location that minimizes the sum of distances to all cities?
- Denote the locations of the cities as $y^{(1)}, \ldots, y^{(m)}$
- Write as the optimization problem:

$$\underset{x}{\text{minimize}} \sum_{i=1}^{m} \|x - y^{(m)}\|_2$$



Continuous Search as Local Search

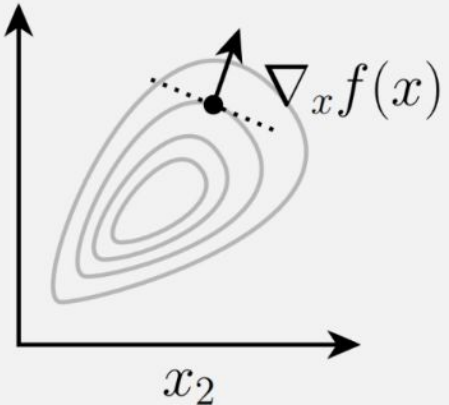
- Discretize it!!!
  - Define a grid with increment δ, use any of the discrete algorithms

- Choose random perturbations to the state
  - First-choice hill-climbing: keep trying until something improves the state
  - Simulated annealing

The Gradient

- A key concept in solving optimization problems is the notation of the gradient of a function (multivariate analogue of derivative)
- For $f: \mathbb{R}^n \to \mathbb{R}$, gradient is defined as vector of partial derivatives
- Points in “steepest direction” of increase in function f

$$\nabla_x f(x) \in \mathbb{R}^n = \begin{bmatrix} \frac{\partial f(x)}{\partial x_1} \\ \frac{\partial f(x)}{\partial x_2} \\ \vdots \\ \frac{\partial f(x)}{\partial x_n} \end{bmatrix}$$



Gradient Descent Algorithm

Gradient motivates a simple algorithm for minimizing f(x): take small steps in the direction of the negative gradient

Algorithm: Gradient Descent
Given:

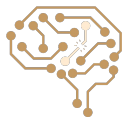
Function f, initial point x_0, step size $\alpha > 0$

Initialize:

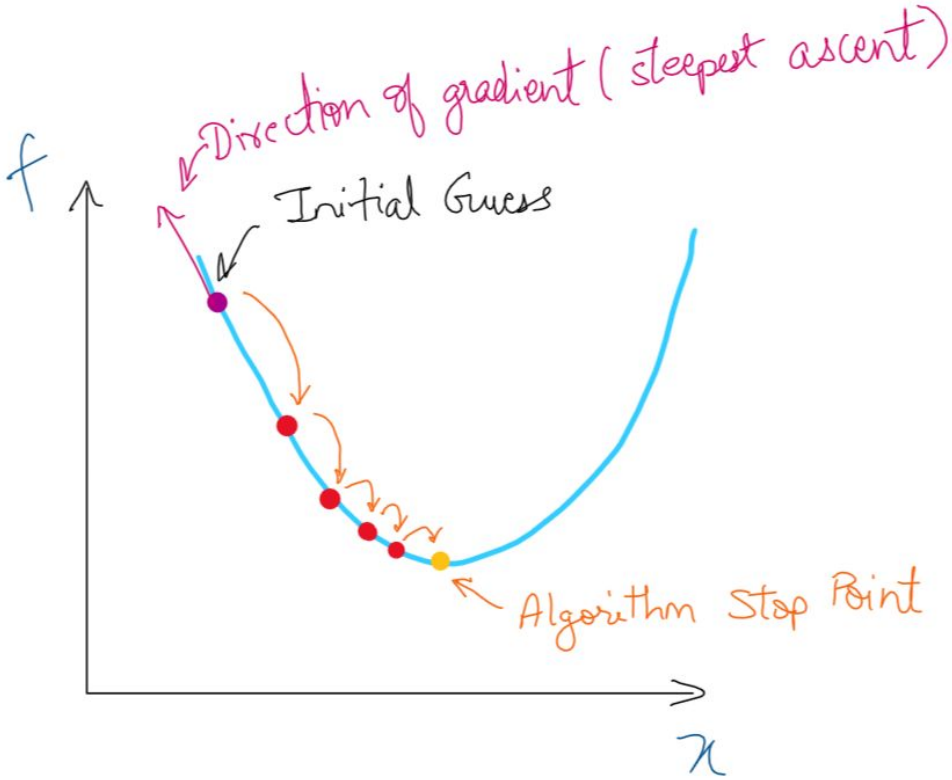
$x \leftarrow x_0$

Repeat until convergence:

$x \leftarrow x - \alpha \nabla_x f(x)$



Gradient Descent Algorithm (cont.)



Gradient Descent Algorithm: One Step Example

$$f(x_1, x_2) = x_1^2 - x_1 x_2 \quad , \quad \alpha = 1 \quad , \quad \text{initial point} = \begin{bmatrix} 1 \\ 2 \end{bmatrix}$$

$$\nabla f(x_1, x_2) = \begin{bmatrix} 2x_1 - x_2 \\ -x_1 \end{bmatrix} \quad , \quad x_{new} \leftarrow x_{old} - \alpha \nabla f$$

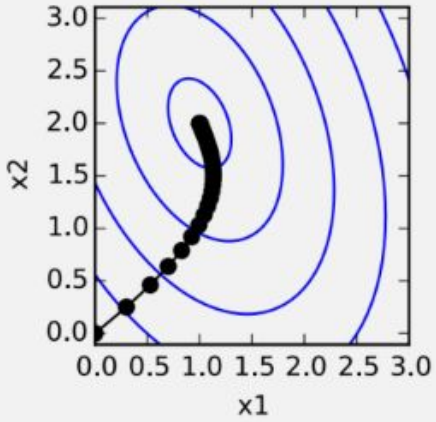
$$x_{new} = \begin{bmatrix} 1 \\ 2 \end{bmatrix} - 1 \times \begin{bmatrix} 2 - 2 \\ -1 \end{bmatrix} = \begin{bmatrix} 1 \\ 3 \end{bmatrix}$$

$$f(1, 2) = 1^2 - 1 \times 2 = -1 \quad , \quad f(1, 3) = 1^2 - 1 \times 3 = -2$$

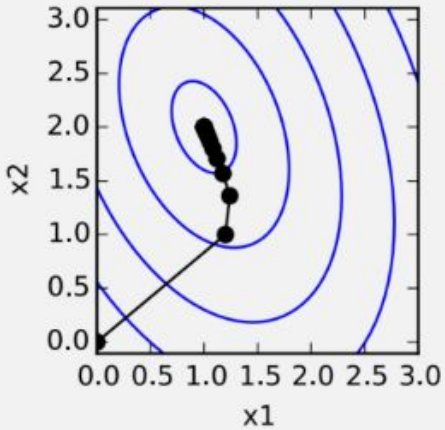
Gradient descent in Practice

Choice of α matters a lot in practice:

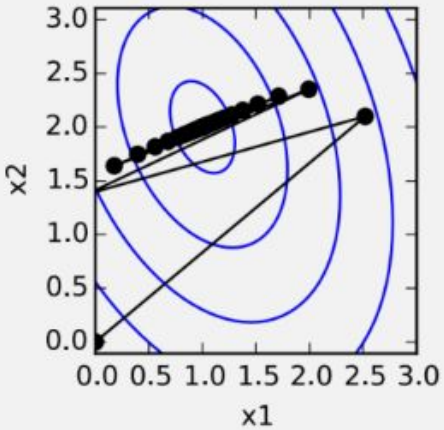
$$\underset{x}{\text{minimize}} \;\; 2x_1^2 + x_2^2 + x_1 x_2 - 6x_1 - 5x_2$$



$\alpha = 0.05$



$\alpha = 0.2$



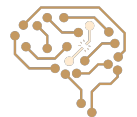
$\alpha = 0.42$

Constrained Optimization

Optimization problems:

$$\min_{x} f(x)$$
$$\text{subject to } x \in \mathcal{C}$$

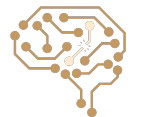
It means that we want to find the value of x that achieves the smallest possible value of f(x), out of all points in C



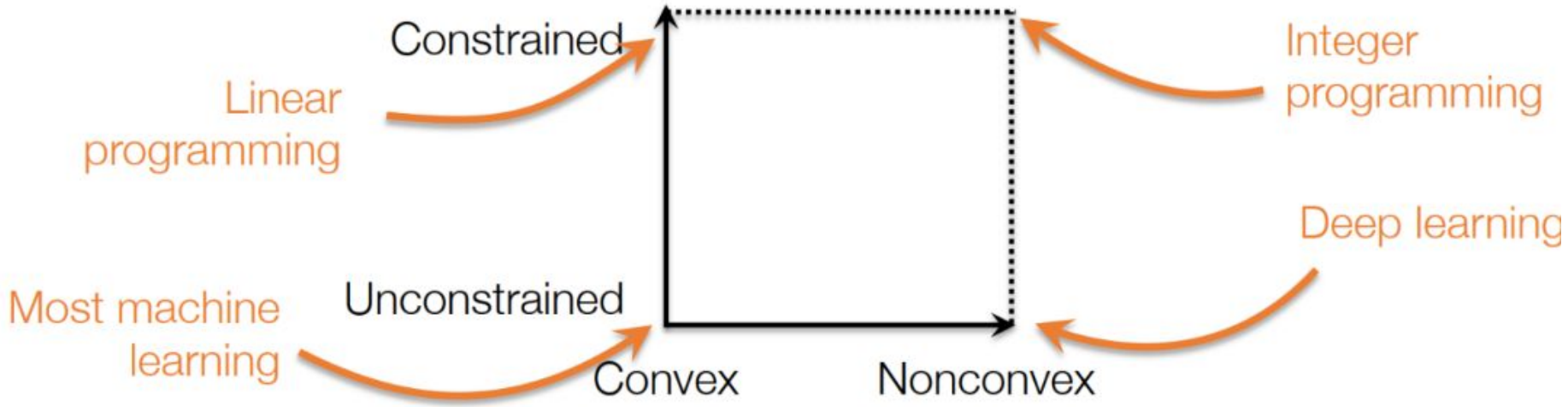
Constrained Optimization (cont.)

Important terms:

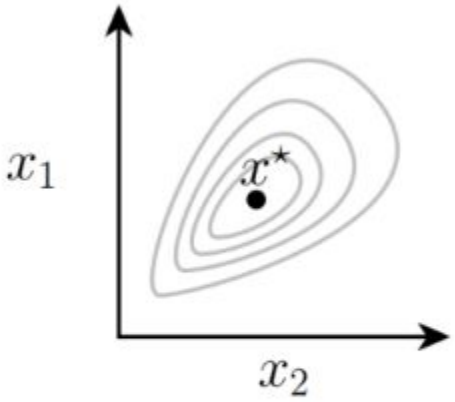
- $x \in \mathbb{R}^n$ – optimization variable (vector with n real-valued entries)
- $f: \mathbb{R}^n \to \mathbb{R}$ – optimization objective
- $\mathcal{C} \subseteq \mathbb{R}^n$ – constraint set
- $x^\star \equiv \underset{x \in \mathcal{C}}{\operatorname{argmin}} f(x)$ – optimal solution
- $f^\star \equiv f(x^\star) \equiv \min_{x \in \mathcal{C}} f(x)$ – optimal objective



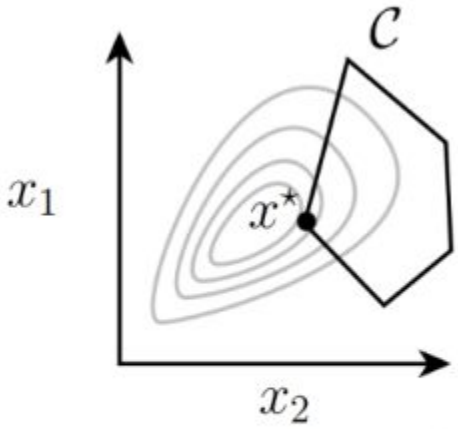
Classes of optimization problems



Constrained vs. Unconstrained



$$\underset{x}{\text{minimize}} \quad f(x)$$

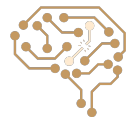


$$\begin{aligned} \underset{x}{\text{minimize}} \quad & f(x) \\ \text{subject to} \quad & x \in \mathcal{C} \end{aligned}$$

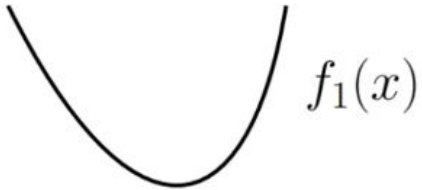
Convex vs. Nonconvex Optimization

Convex function

Nonconvex function



Convex vs. nonconvex optimization



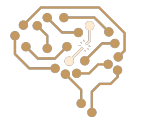
Convex function

Nonconvex function

- Convex problem:

$$\min_x f(x)$$
$$\text{subject to } x \in \mathcal{C}$$

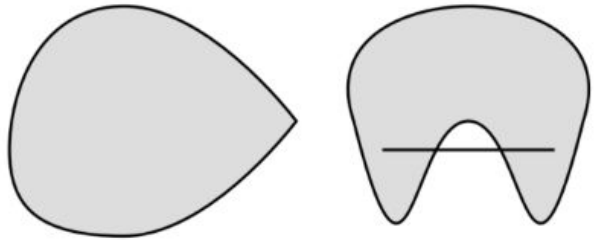
- Where f is a convex function and $\mathcal{C}$ is a convex set



Convex sets

A set $\mathcal{C}$ is convex if, for any $x, y \in \mathcal{C}$ and $0 \leq \theta \leq 1$

$$\theta x + (1-\theta)y \in \mathcal{C}$$



Convex set **Nonconvex set**

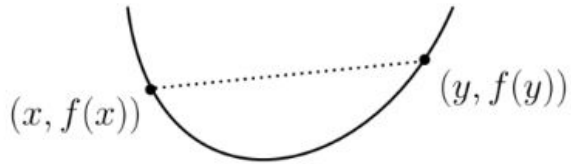
Examples:

- All points $\mathcal{C} = \mathbb{R}^n$
- Intervals $\mathcal{C} = \{x \in \mathbb{R}^n | \ l \leq x \leq u\}$ (elementwise inequality)
- Linear equalities $\mathcal{C} = \{x \in \mathbb{R}^n | Ax = b\}$ (for $A \in \mathbb{R}^{m \times n}, b \in \mathbb{R}^m$)
- Intersection of convex sets $\mathcal{C} = \bigcap_{i=1}^{m} \mathcal{C}_i$

Convex functions

A function $f: \mathbb{R}^n \to \mathbb{R}$ is convex if, for any $x, y \in \mathbb{R}^n$ and $0 \leq \theta \leq 1$

$$f(\theta x + (1-\theta)y) \leq \theta f(x) + (1-\theta) f(y)$$



Convex functions “curve upwards” (or at least not downwards)

If f is convex then $-f$ is concave

If f is both convex and concave, it is affine, must be of form:

$$f(x) = \sum_{i=1}^{n} a_i x_i + b$$

Examples of convex functions

Exponential: $f(x) = \exp(ax), a \in \mathbb{R}$

Negative logarithm: $f(x) = -\log x$, with domain $x > 0$

Squared Euclidean norm: $f(x) = \|x\|_2^2 \equiv x^T x \equiv \sum_{i=1}^n x_i^2$

Euclidean norm: $f(x) = \|x\|_2$

Convex function of affine function $f(x) = g(Ax + b)$, g convex

Non-negative weighted sum of convex functions

$$f(x) = \sum_{i=1}^{m} w_i f_i(x), \qquad w_i \geq 0, f_i \text{ convex}$$

Convex Optimization

- The key aspect of convex optimization problems that make them tractable is that all local optima are global optima
- **Definition**: a point x is globally optimal if x is feasible and there is no feasible y such that $f(y) < f(x)$
- **Definition**: a point x is locally optimal if x is feasible and there is some $R > 0$ such that for all feasible y with $\|x - y\|_2 \leq R, f(x) \leq f(y)$
- Theorem: for a convex optimization problem all locally optimal points are globally optimal

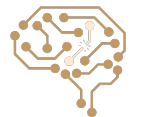
Gradient Descent Disadvantages

- Local minima problem
- However, when objective function is convex, all local minima are also global minima
- In this case, gradient descent can converge to the global solution.

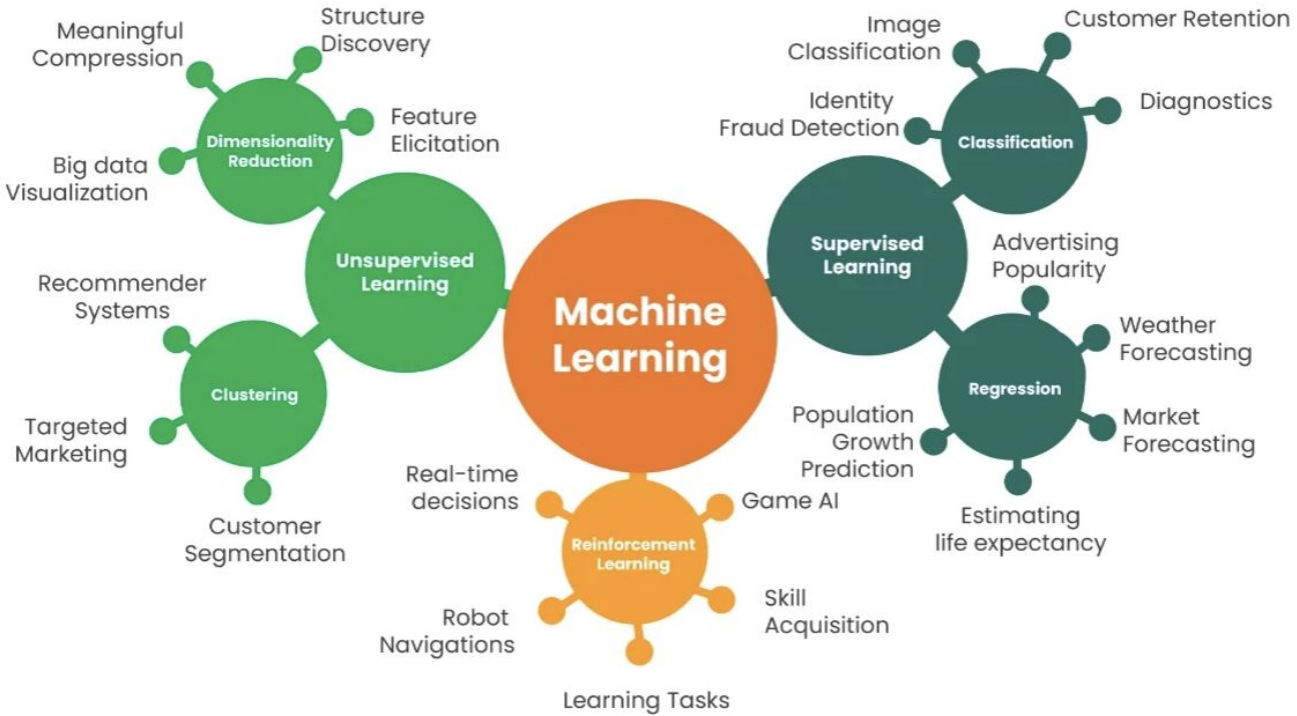
Concepts of Machine Learning

Types of ML problems

- Supervised learning
  - Given: training data + desired outputs (labels/targets)
- Unsupervised learning
  - Given: training data (without desired outputs)
- Reinforcement learning
  - Rewards from sequence of actions

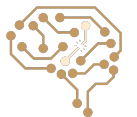


Types of ML problems (Cont.)



Components of Supervised Learning

- Input space: $\mathcal{X}$
- Output space: $\mathcal{Y}$
- Unknown target function: $f : \mathcal{X} \rightarrow \mathcal{Y}$
- Training data: $(\boldsymbol{x}_1, y_1)$, $(\boldsymbol{x}_2, y_2)$,...,$(\boldsymbol{x}_N, y_N)$
- Pick a formula $g : \mathcal{X} \rightarrow \mathcal{Y}$ that approximates the target function f
- Hypothese Set $\mathcal{H}$: a set (class) of functions that g comes from



Supervised Learning: Regression vs. Classification

Regression: predict a continuous target variable

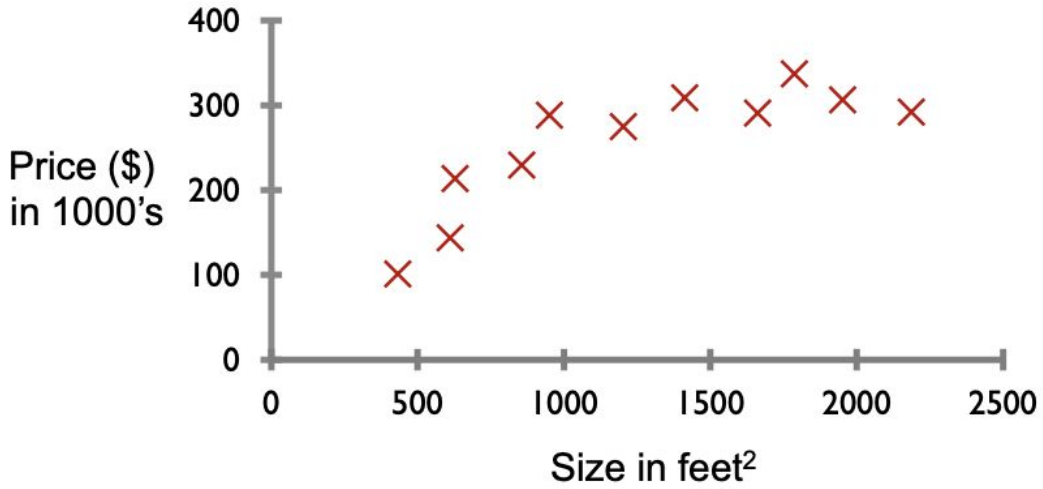
E.g., $y \in [0,1]$

Classification: predict a discrete target variable

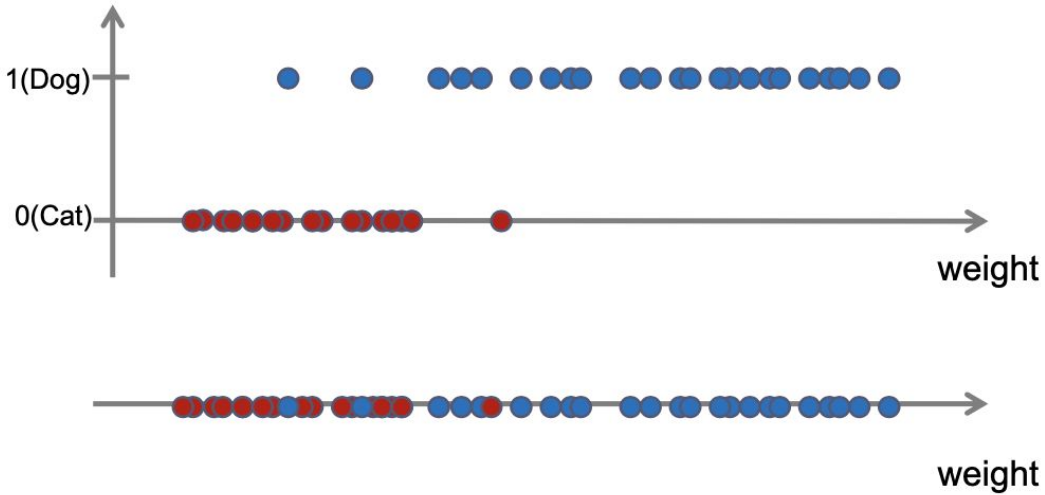
E.g.,$y \in \{1,2,\ldots,C\}$

Regression Example

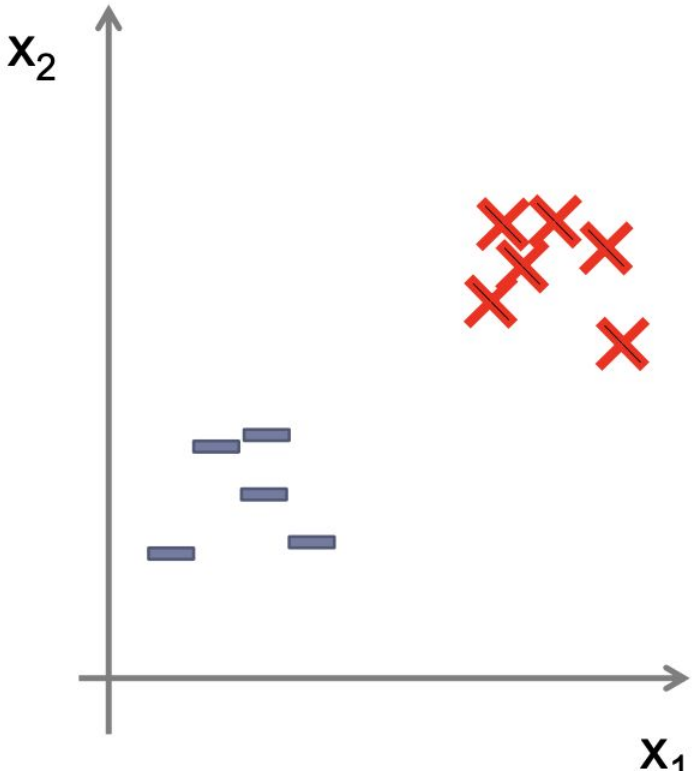
- Housing price prediction



Classification Example



Classification Example (cont.)



Training data

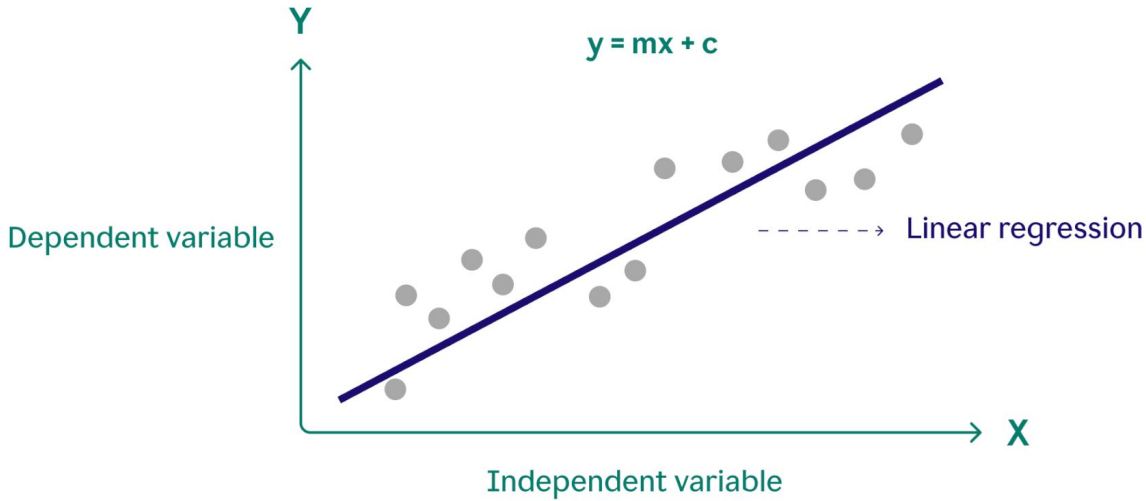
x_1	x_2	y	
0.9	2.3	1	−
3.5	2.6	1	−
2.6	3.3	1	−
2.7	4.1	1	−
1.8	3.9	1	−
6.5	6.8	-1	×
7.2	7.5	-1	×
7.9	8.3	-1	×
6.9	8.3	-1	×
8.8	7.9	-1	×
9.1	6.2	-1	×

Baselines

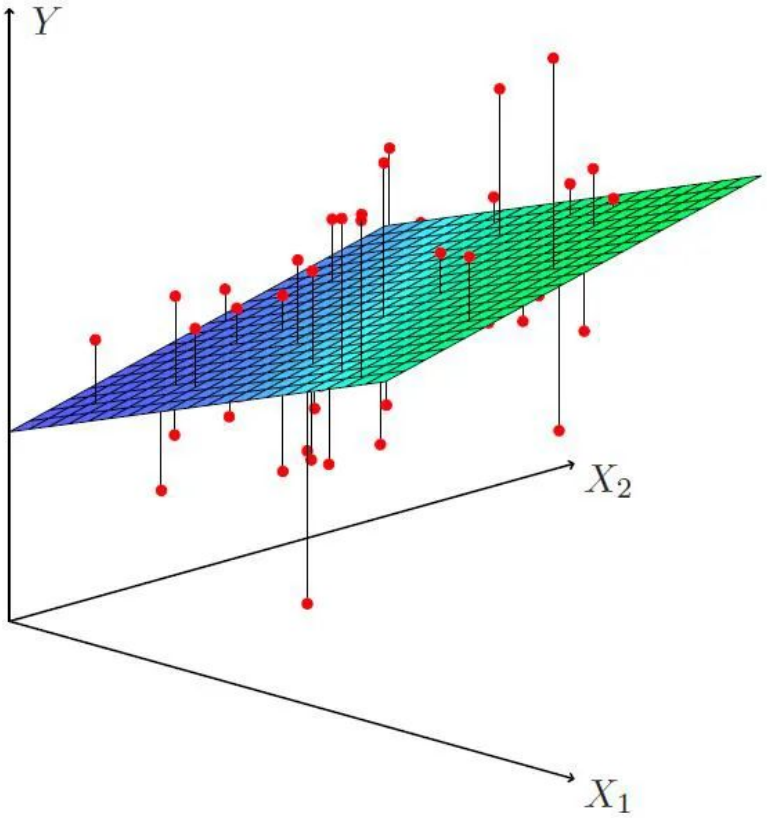
- Baselines are very simple “straw man” procedures
- Help determine how hard the task and what a “good” accuracy are
- A simple (weak) baseline: most frequent label classifier
- Gives all test instances the most common label in the training set
- E.g. for spam filtering, might label everything as ham
- Accuracy might be very high if the problem is skewed
- E.g. calling everything “ham” gets 66%, so a classifier that gets 70% isn’t very good…

For real research, usually use previous work as a (strong) baseline

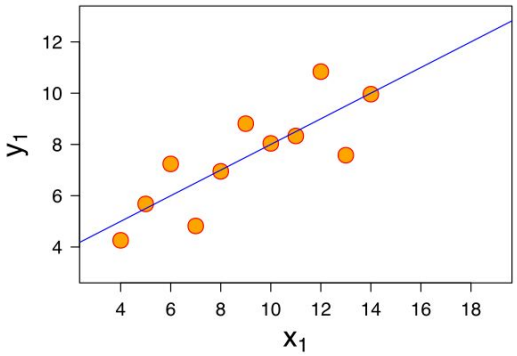
Linear Regression: 1 Feature

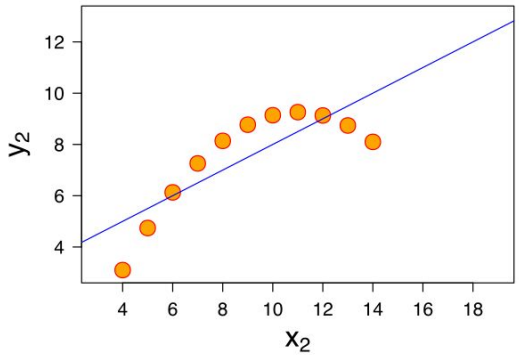


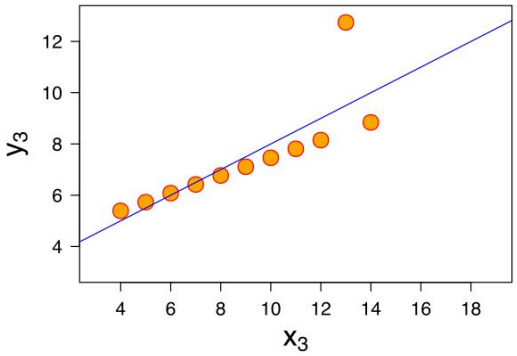
Linear Regression: 2 Features

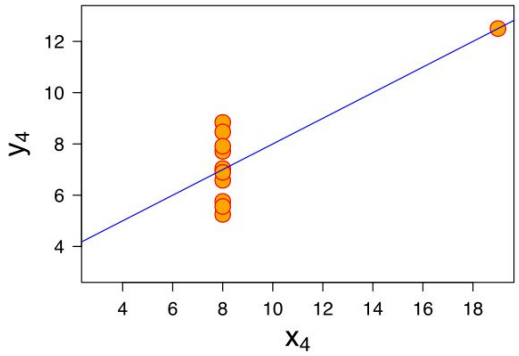


Linear Regression Examples

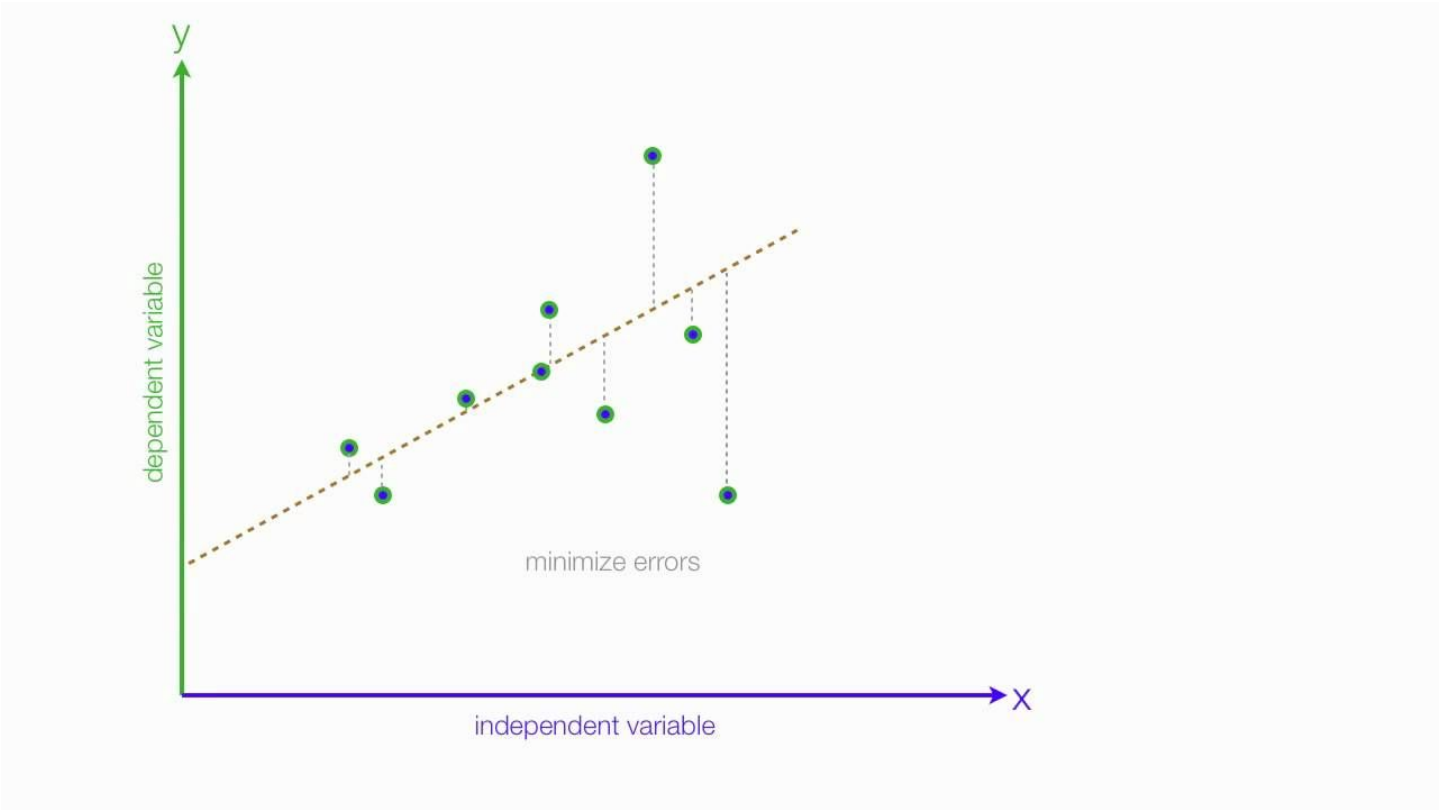




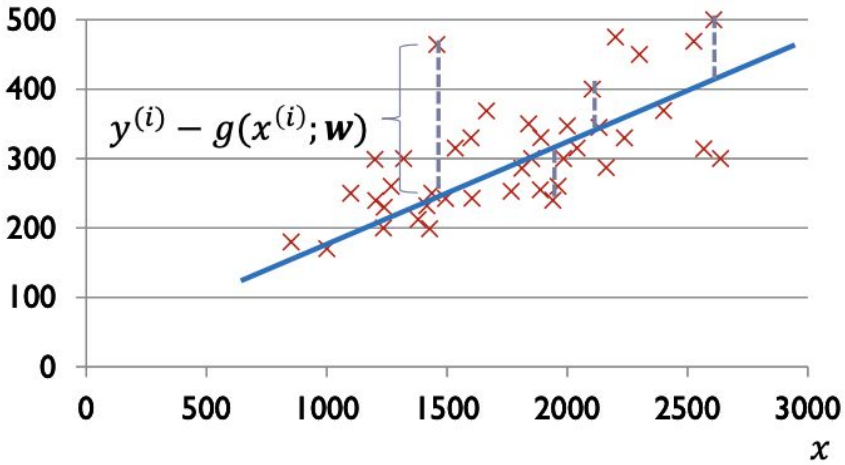




Linear Regression & Error



Linear regression

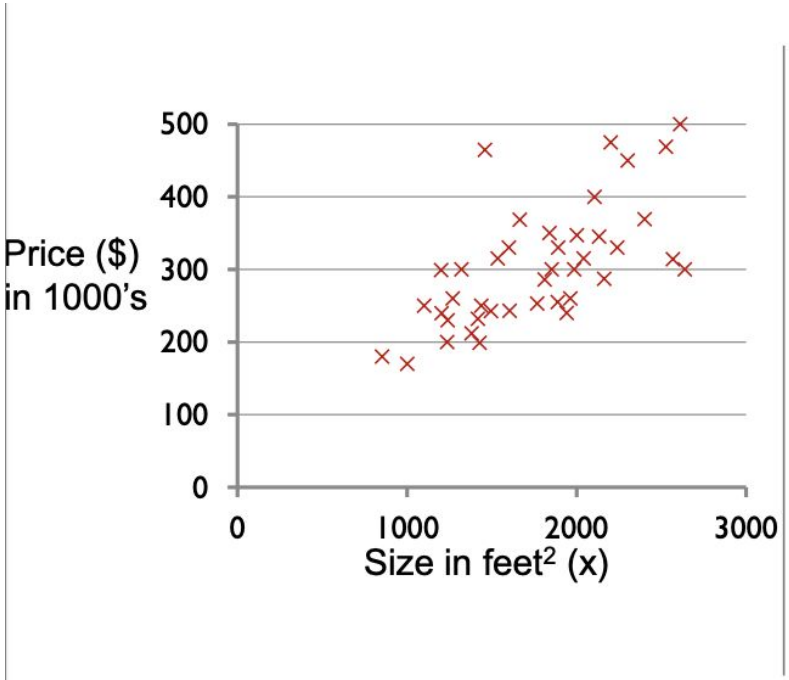


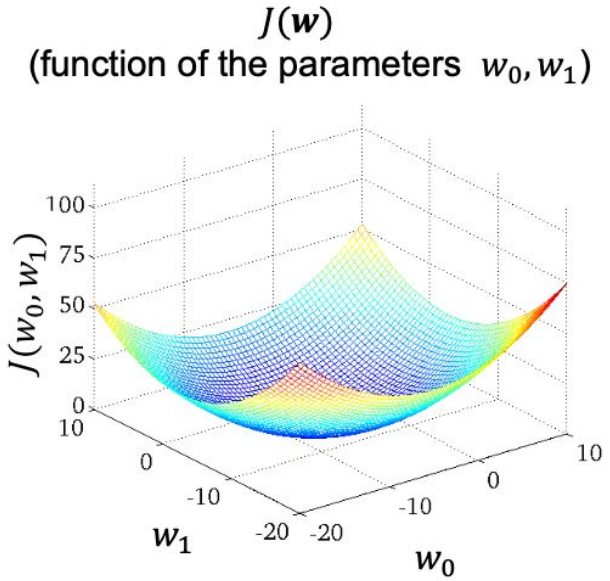
Cost function:

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} \left(y^{(i)} - g(x^{(i)}; \boldsymbol{w})\right)^2$$

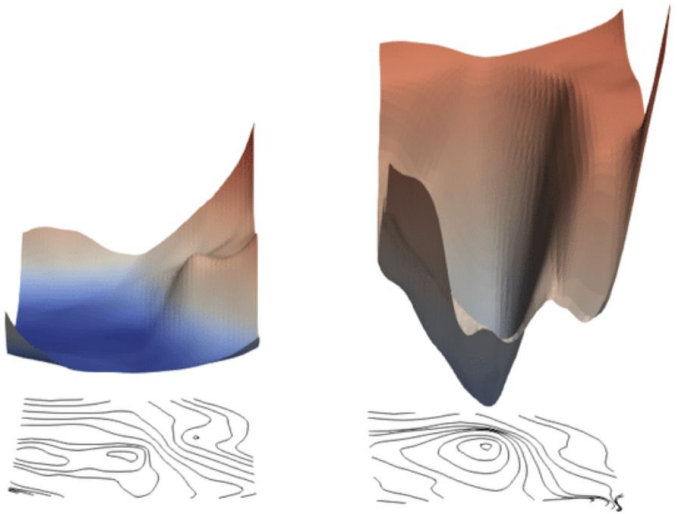
$$= \sum_{i=1}^{n} \left(y^{(i)} - w_0 - w_1 x^{(i)}\right)^2$$

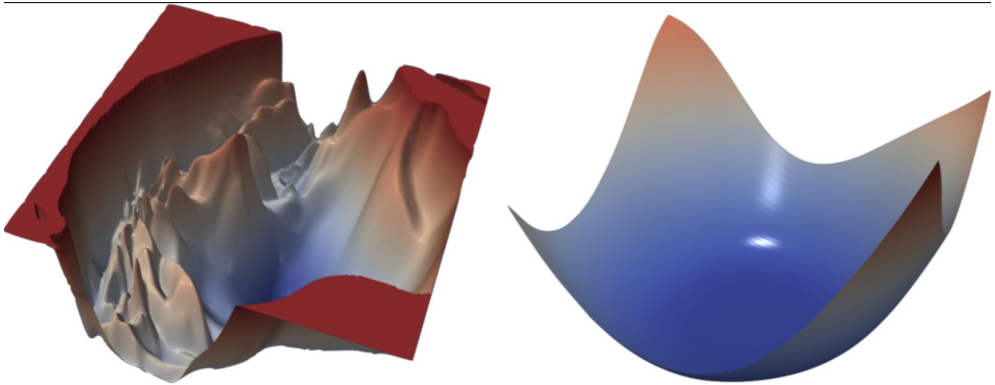
Cost function





Loss Landscape





(a) ResNet-110, no skip connections

(b) DenseNet, 121 layers

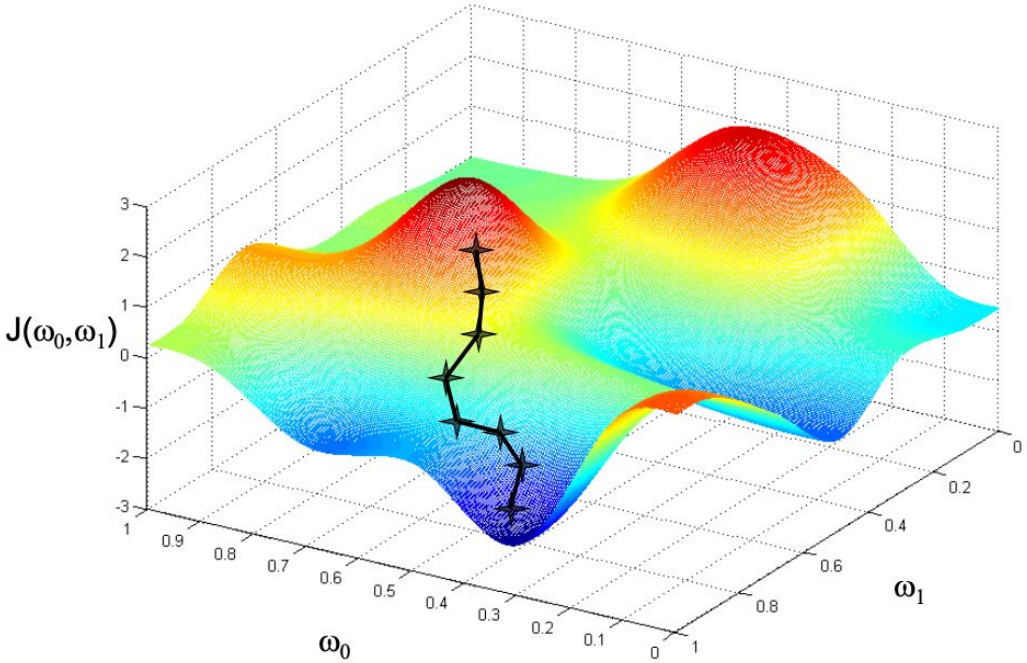
Gradient descent

- Optimization algorithm to find $\boldsymbol{w}^* = \underset{\boldsymbol{w}}{\operatorname{argmin}} J(\boldsymbol{w})$
- Gradient descent: In each step, takes steps proportional to the negative of the gradient vector of the function at the current point:

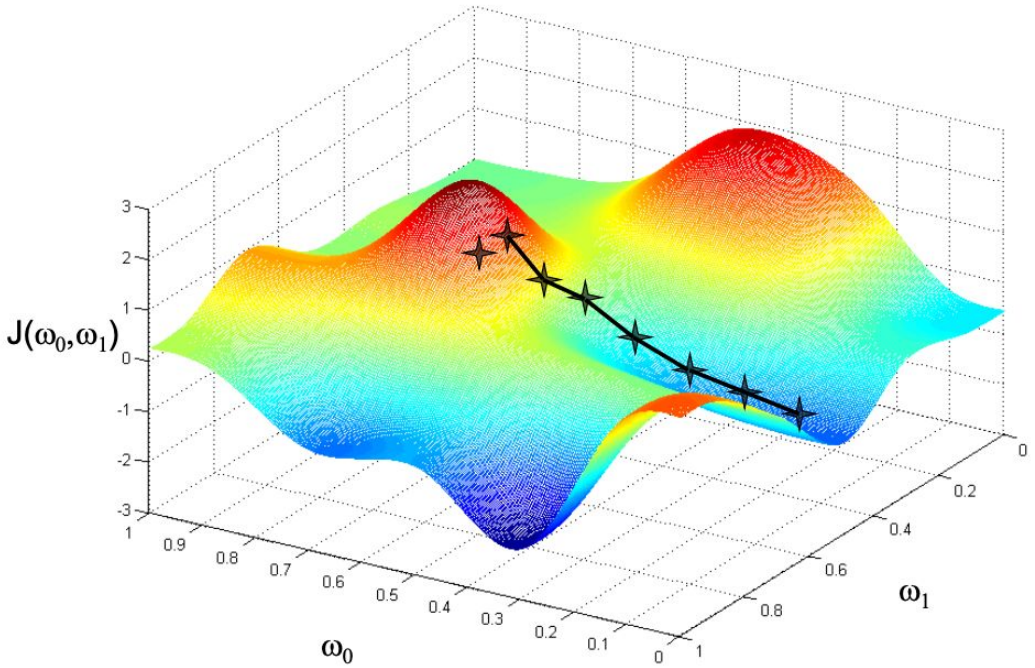
$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^t - \gamma_t \nabla J(\boldsymbol{w}^t)$$

- $J(w)$ decreases fastest if one goes from w^t in the direction of $-\nabla J(wt)$
- Assumption: $J(w)$ is defined and differentiable in a neighborhood of a point w^t

Gradient Descent with Non-convex Cost Functions



Gradient Descent with Non-convex Cost Functions (cont.)



Gradient descent

Minimize $J(\boldsymbol{w})$

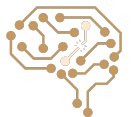
Step size (Learning rate parameter)

$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^{t} - \eta \nabla_{\boldsymbol{w}} J(\boldsymbol{w}^{t})$$

$$\nabla_{\boldsymbol{w}} J(\boldsymbol{w}) = \left[\frac{\partial J(\boldsymbol{w})}{\partial w_1}, \frac{\partial J(\boldsymbol{w})}{\partial w_2}, \ldots, \frac{\partial J(\boldsymbol{w})}{\partial w_d} \right]^T$$

If η is small enough, then $J(\boldsymbol{w}^{t+1}) \leq J(\boldsymbol{w}^{t})$.

η can be allowed to change at every iteration as η_t.



Gradient descent for SSE cost function

Minimize $J(\boldsymbol{w})$

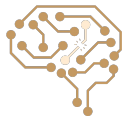
$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^{t} - \eta \nabla_{\boldsymbol{w}} J(\boldsymbol{w}^{t})$$

$J(\boldsymbol{w})$: Sum of squares error

$$J(\boldsymbol{w}) = \sum_{i=1}^{n} \left(y^{(i)} - g\left(\boldsymbol{x}^{(i)}; \boldsymbol{w}\right) \right)^2$$

Weight update rule for $g(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x}$: $\boldsymbol{w} = \begin{bmatrix} w_0 \\ w_1 \\ \vdots \\ w_d \end{bmatrix}$ $\boldsymbol{x} = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_d \end{bmatrix}$

$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^{t} + \eta \sum_{i=1}^{n} \left(y^{(i)} - \boldsymbol{w}^{t^T} \boldsymbol{x}^{(i)} \right) \boldsymbol{x}^{(i)}$$



Gradient descent for SSE cost function

Weight update rule: $g(\boldsymbol{x}; \boldsymbol{w}) = \boldsymbol{w}^T \boldsymbol{x}$

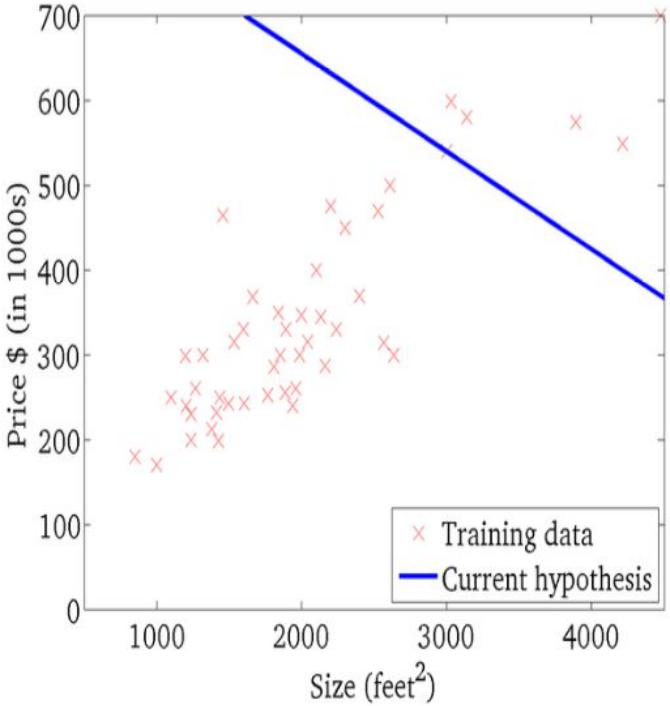
$$\boldsymbol{w}^{t+1} = \boldsymbol{w}^t + \eta \sum_{i=1}^{n} \left(y^{(i)} - \boldsymbol{w}^T \boldsymbol{x}^{(i)}\right) \boldsymbol{x}^{(i)}$$

Batch mode: each step considers all training data

η: too small → gradient descent can be slow.

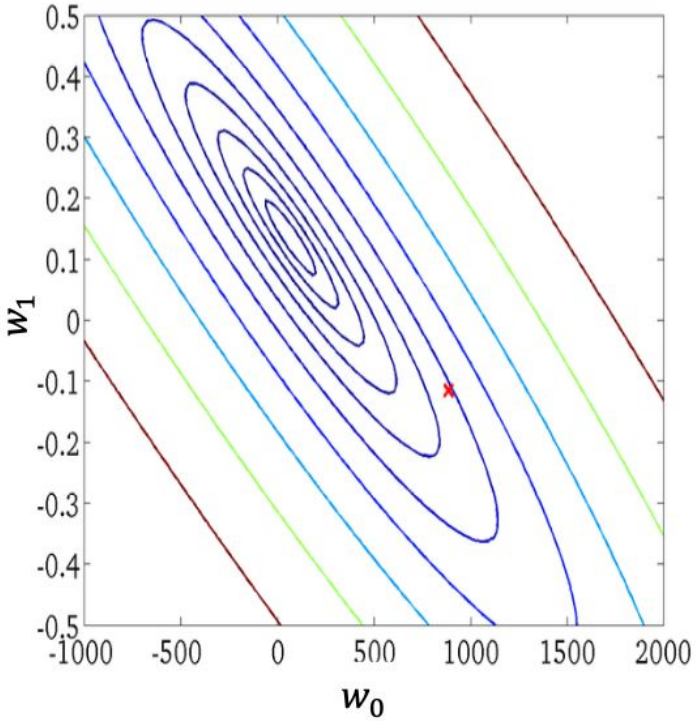
η: too large → gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

$g(x; w_0, w_1) = w_0 + w_1 x$

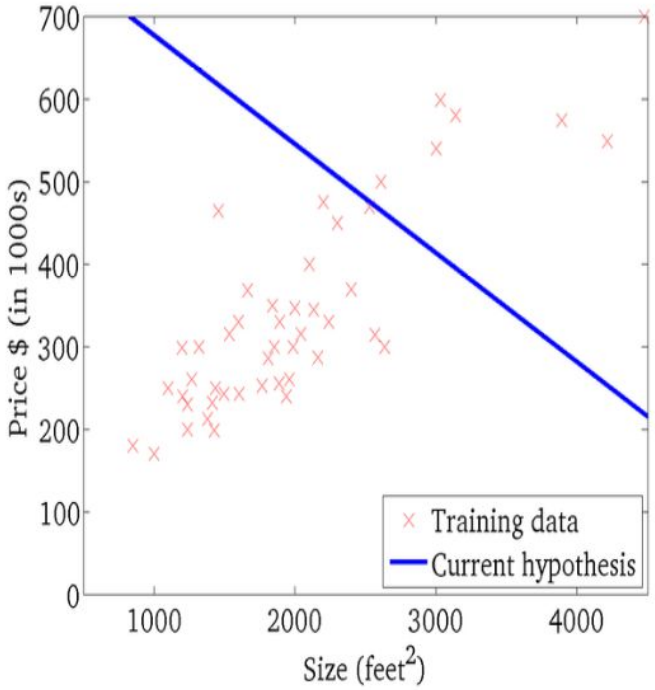


$J(w_0, w_1)$

(function of the parameters w_0, w_1)

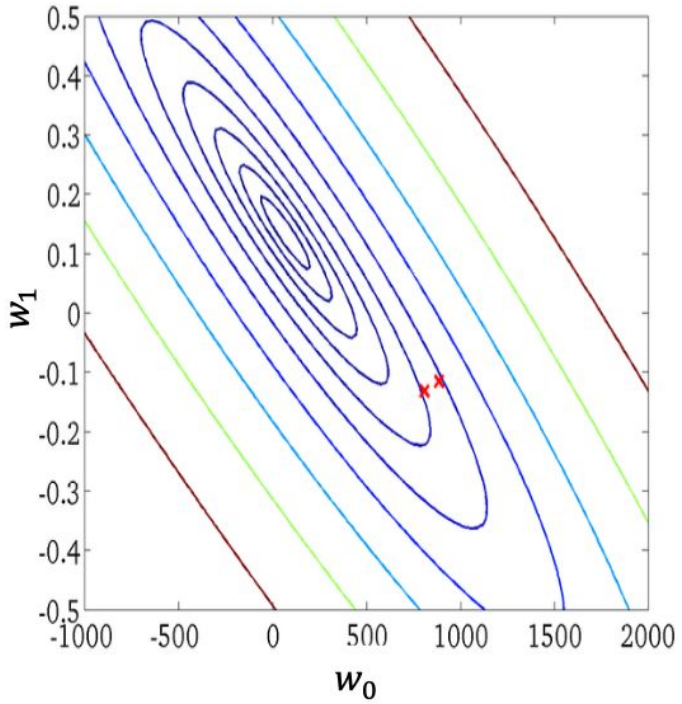


$g(x; w_0, w_1) = w_0 + w_1 x$

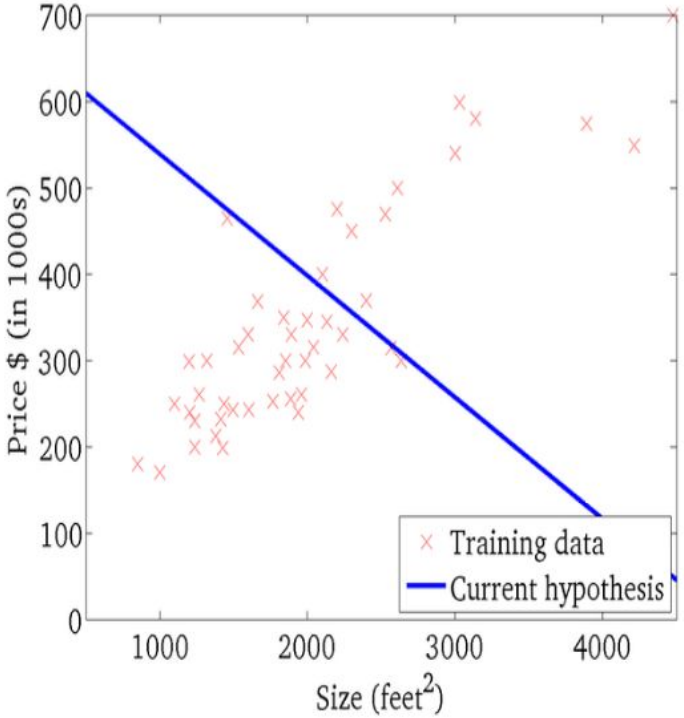


$J(w_0, w_1)$

(function of the parameters w_0, w_1)

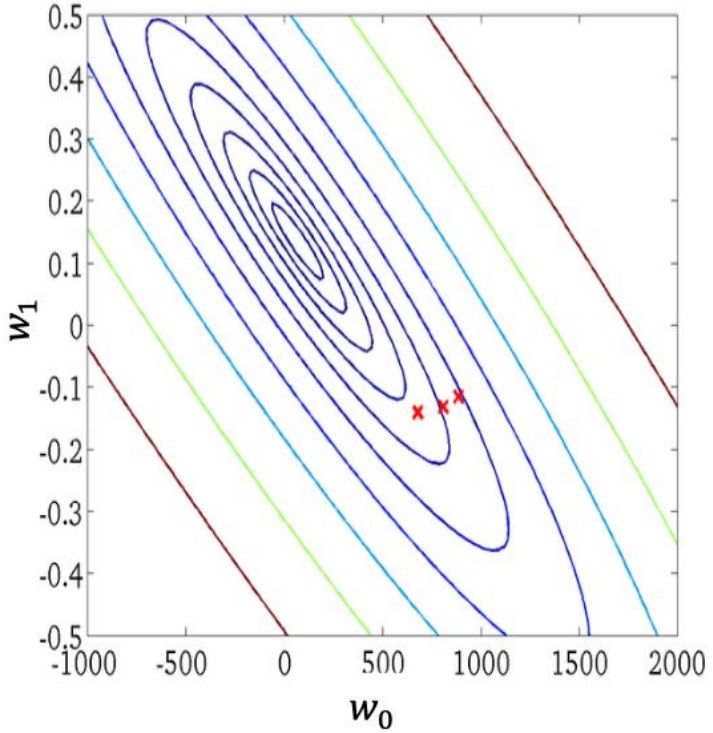


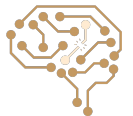
$g(x; w_0, w_1) = w_0 + w_1 x$



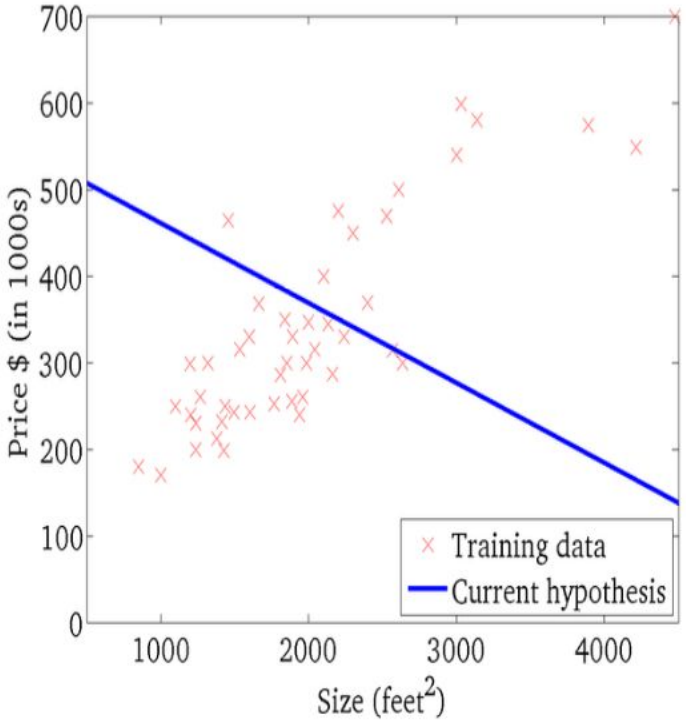
$J(w_0, w_1)$

(function of the parameters w_0, w_1)



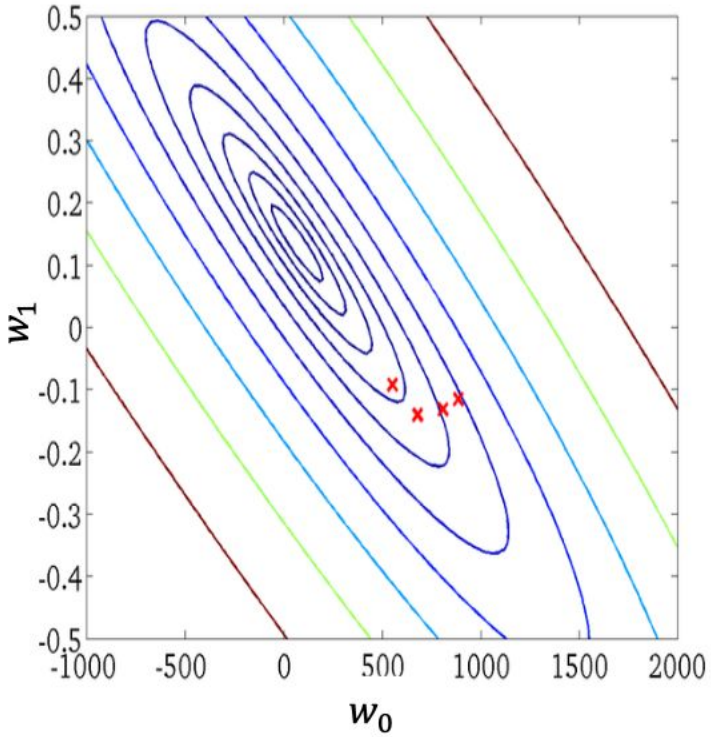


$g(x; w_0, w_1) = w_0 + w_1 x$

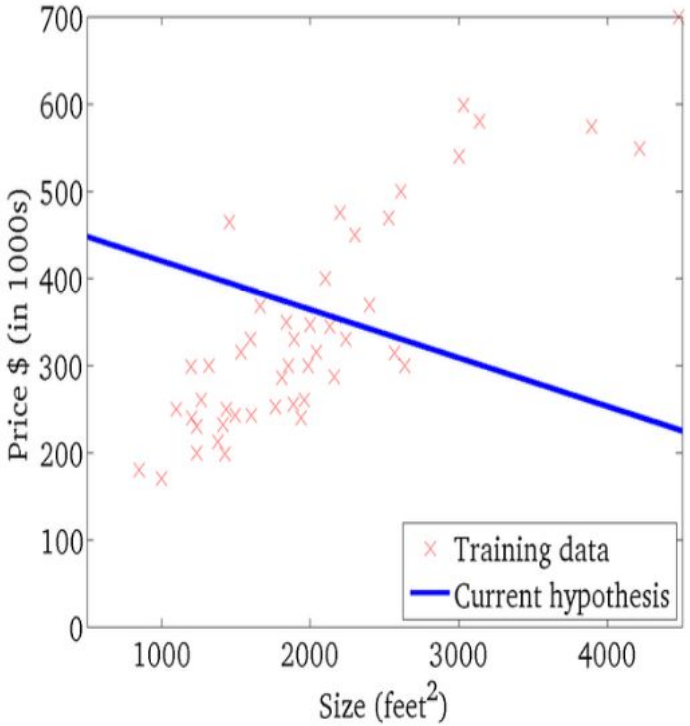


$J(w_0, w_1)$

(function of the parameters w_0, w_1)

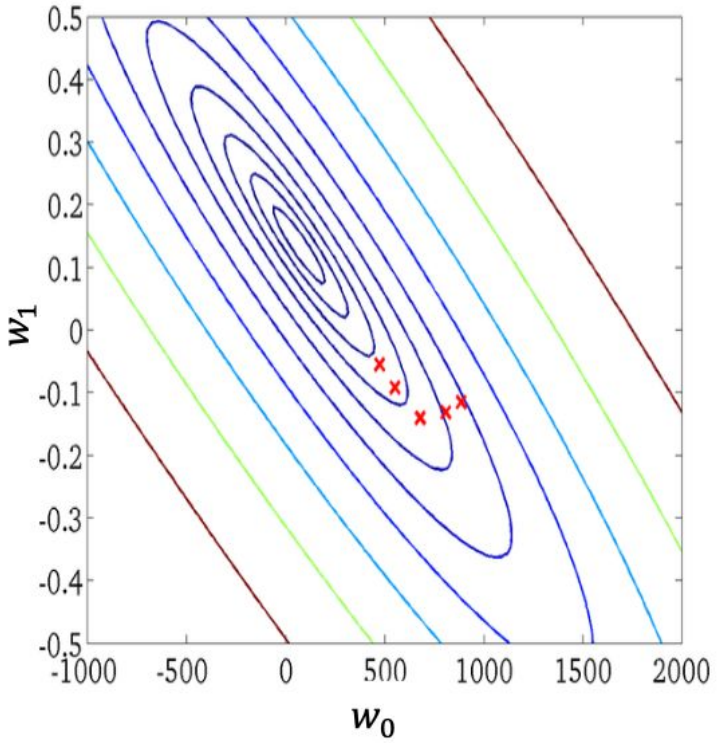


$$g(x; w_0, w_1) = w_0 + w_1 x$$

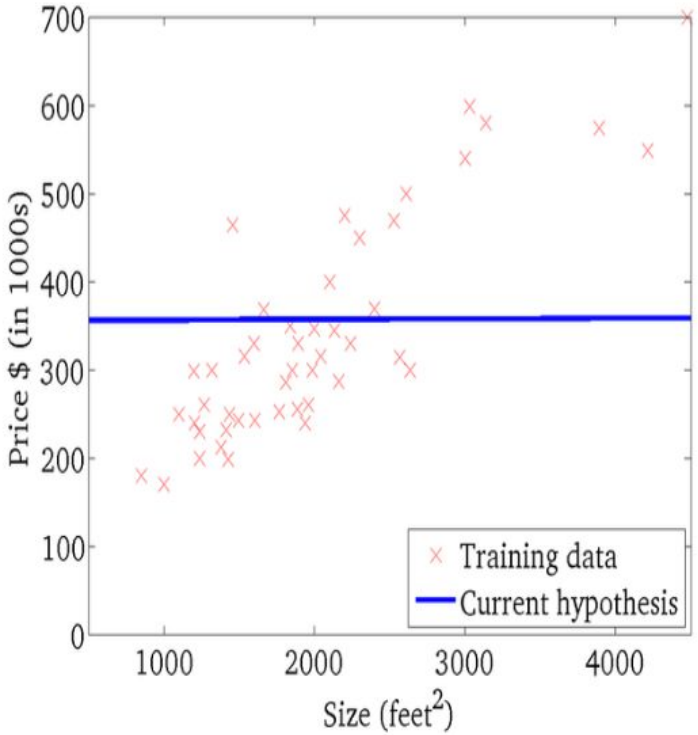


$$J(w_0, w_1)$$

(function of the parameters w_0, w_1)

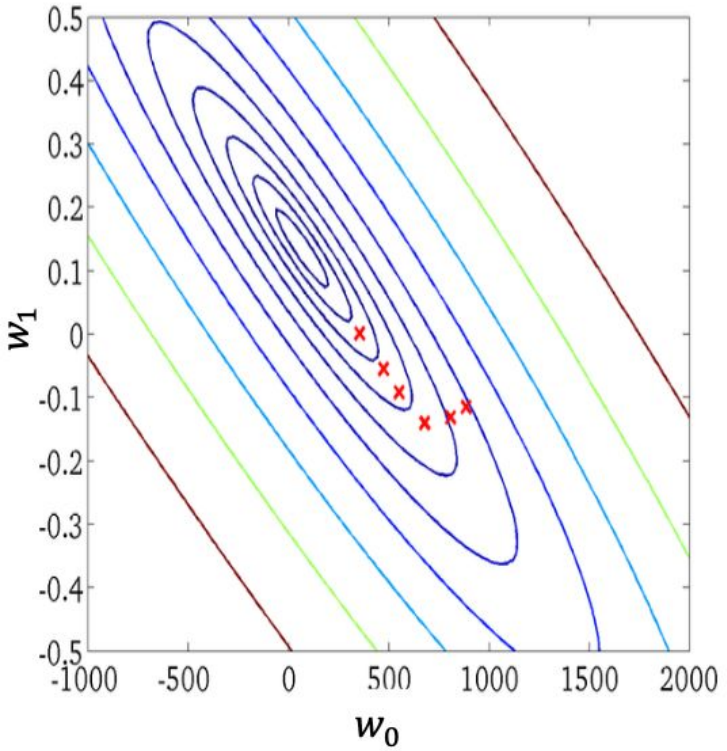


$g(x; w_0, w_1) = w_0 + w_1 x$

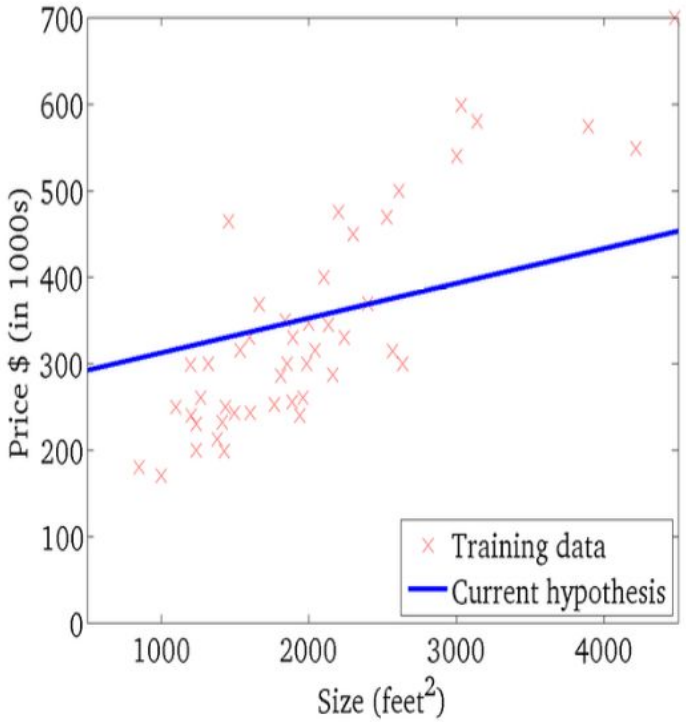


$J(w_0, w_1)$

(function of the parameters w_0, w_1)

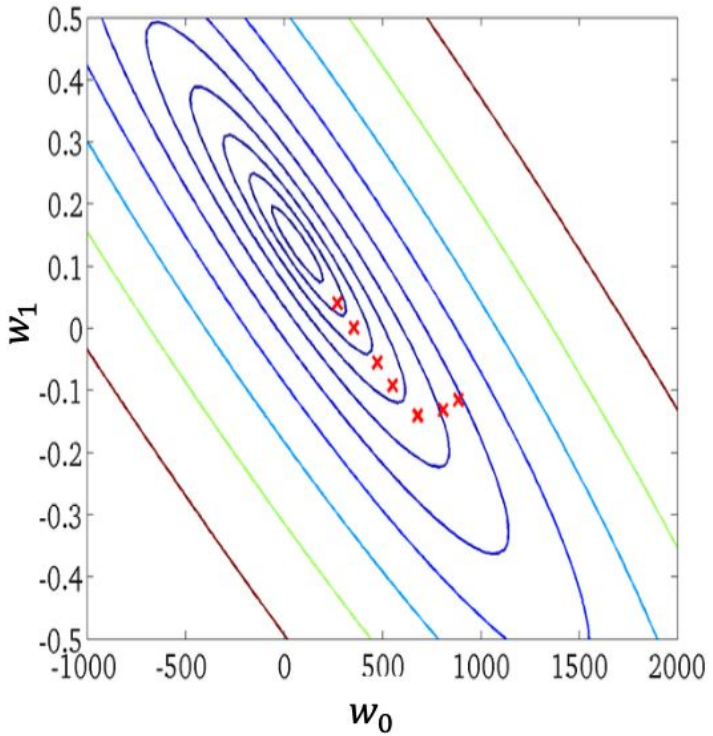


$$g(x; w_0, w_1) = w_0 + w_1 x$$

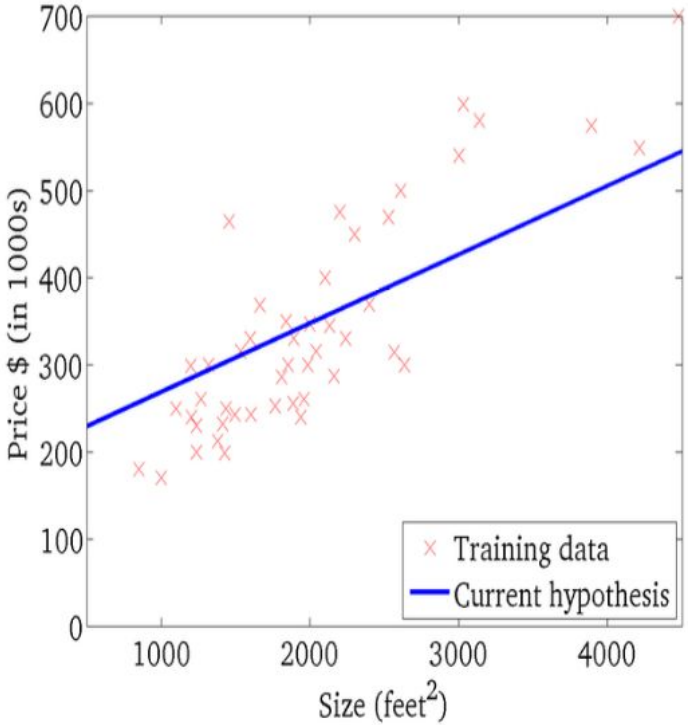


$$J(w_0, w_1)$$

(function of the parameters w_0, w_1)

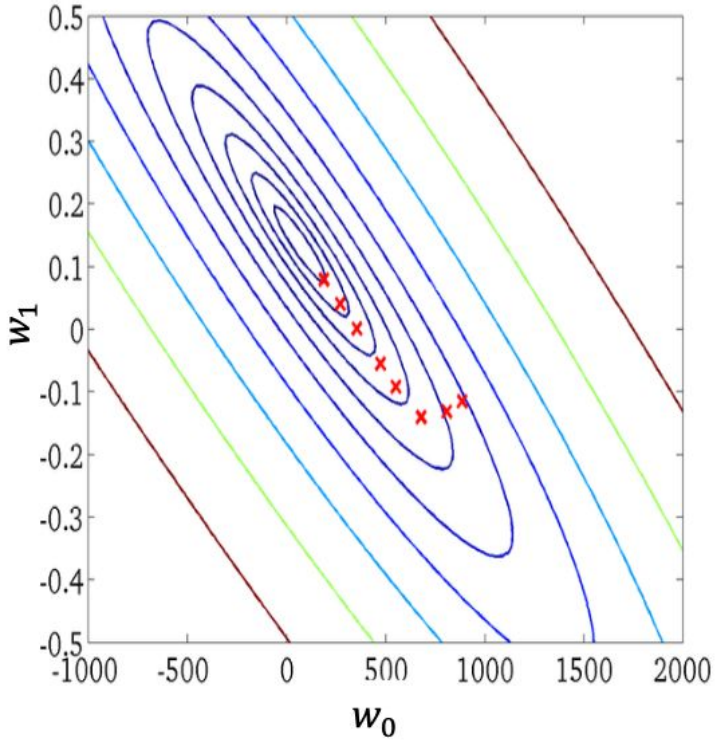


$g(x; w_0, w_1) = w_0 + w_1 x$

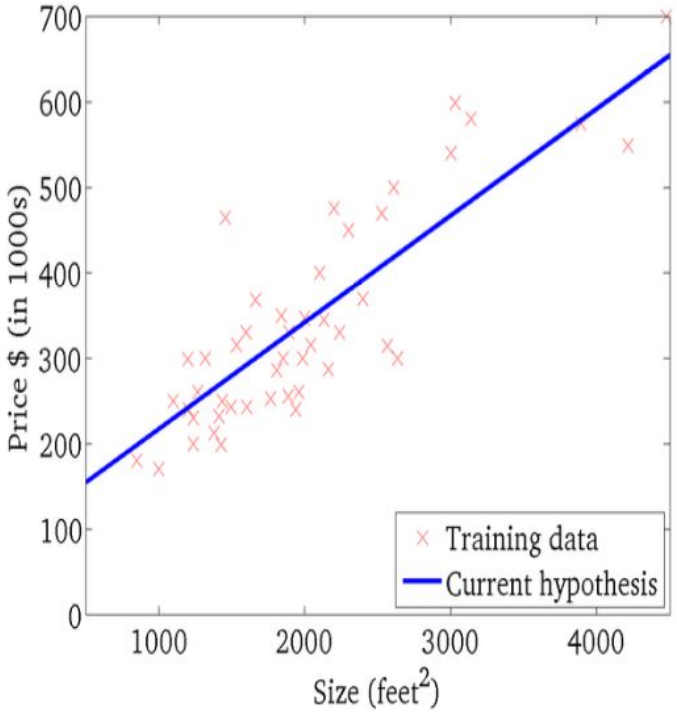


$J(w_0, w_1)$

(function of the parameters w_0, w_1)



$g(x; w_0, w_1) = w_0 + w_1 x$



$J(w_0, w_1)$

(function of the parameters w_0, w_1)